

Programming II

Object Oriented Programming (OOP) **More about working with Objects**

More about working with Objects

- **Passing**
- **Returning**
- **Comparing**
- **Copying**

1- Passing Objects as Arguments:

- Objects can be passed to methods as arguments.
- Java passes all arguments by value.
- When an object is passed as an argument, the value of the reference variable is passed.
- The value of the reference variable is an address or reference to the object in memory.
- A copy of the object is not passed, just a pointer to the object.
- When a method receives a reference variable as an argument, it is possible for the method to modify the contents of the object referenced by the variable.

Call (Pass) by Value:

```
public class Rectangle{  
    private double length;  
    private double width;  
    public Rectangle ()  
    { length = 10;  
      width = 15;  
    }  
    public void setlength (double l)  
    {  
        length = l;  
    }  
    public void setwidth (double w)  
    {  
        width = w;  
    }  
}
```

Call (Pass) by Value:

```
public double getlength()
{
    return length;
}
public double getwidth()
{
    return width;
}
public double getarea()
{
    return length*width;
}
}
```

Call (Pass) by Value:

```
public class Main
{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle ();
        double rectangle_length = 60;
        r1.setlength(rectangle_length);
        System.out.println(rectangle_length);
    }
}
```

60.0

Call (Pass) by Reference:

```
public class Rectangle{
    private double length;
    private double width;
    public Rectangle ()
    { length = 10;
      width = 15;
    }
    public Rectangle (double l , double w)
    {
        length = l;
        width = w;
    }
    public void modify_object (Rectangle r)
    {
        r.length = 77;
        r.width   = 88;
    }
}
```

Call (Pass) by Reference:

```
public class Main
{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle ();
        Rectangle r2 = new Rectangle (50 , 60);
        System.out.println(r2.getlength());
        System.out.println(r2.getwidth());
        r1.modify_object(r2);
        System.out.println("After passing r2 as an argument");
        System.out.println(r2.getlength());
        System.out.println(r2.getwidth());
    }
}
```

50.0

60.0

After passing r2 as an argument

77.0

88.0

2- Returning Objects from Methods:

- Methods are not limited to returning the primitive data types.
- Methods can return references to objects as well.
- Just as with passing arguments, a copy of the object is not returned, only its address.

Example:

Create a method inside the Rectangle class that adds the contents of two objects

```
public class Rectangle{
    private double length;
    private double width;
    public Rectangle ()
    { length = 10;
      width = 15;
    }
    public Rectangle (double l , double w)
    {
        length = l;
        width = w;
    }

    public Rectangle add (Rectangle R1 , Rectangle R2)
    {
        Rectangle result = new Rectangle ();
        result.length = R1.length + R2.length;
        result.width = R1.width + R2.width;
        return result;
    }
}
```

Example:

```
public class Main
{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle (30,60);
        Rectangle r2 = new Rectangle (40,50);
        Rectangle r3 = new Rectangle ();
        Rectangle r4 = r3.add(r1,r2);
        System.out.println("The length of the new rectangle = "+ r4.getlength());
        System.out.println("The width of the new rectangle = "+ r4.getwidth());
    }
}
```

The length of the new rectangle = 70.0

The width of the new rectangle = 110.0

Example:

- **this:** is used to point the current object

```
public class Rectangle{
    private double length;
    private double width;
    public Rectangle ()
    { length = 10;
      width = 15;
    }
    public Rectangle (double l , double w)
    {
        length = l;
        width = w;
    }

    public Rectangle add (Rectangle R2)
    {
        Rectangle result = new Rectangle ();
        result.length = this.length + R2.length;
        result.width   = this.width   + R2.width;
        return result;
    }
}
```

Example:

```
public class Main
{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle (30,60);
        Rectangle r2 = new Rectangle (40,50);
        Rectangle r3 = r1.add(r2);
        System.out.println("The length of the new rectangle = "+ r3.getlength());
        System.out.println("The width of the new rectangle = "+ r3.getwidth());
    }
}
```

```
The length of the new rectangle = 70.0
The width of the new rectangle = 110.0
```

Example:

```
public class Main
{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle (30,60);
        Rectangle r2 = new Rectangle (40,50);
        System.out.println("The length of the new rectangle = "+ r1.add(r2).getlength());
        System.out.println("The width of the new rectangle = "+ r1.add(r2).getwidth());
    }
}
```

The length of the new rectangle = 70.0
The width of the new rectangle = 110.0

3- Comparing between two objects:

Using The ==operator with objects:

Example:

```
public class Main
{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle (30,50);
        Rectangle r2 = new Rectangle (30,50);
        if (r1==r2)    //    The addresses are compared
            System.out.println("The objects are equal");
        else
            System.out.println("The objects are different");
    }
}
```

The objects are different

- Only the addresses of the objects are compared.

Example:

```
public class Rectangle{
    private double length;
    private double width;
    public Rectangle ()
    { length = 10;
      width = 15;
    }
    public Rectangle (double l , double w)
    {
        length = l;
        width = w;
    }
    public boolean isequal (Rectangle r)
    {
        if (this.length == r.length && this.width == r.width)
            return true;
        else
            return false;
    }
}
```


Example:

```
public class Main
{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle (30,50);
        Rectangle r2 = new Rectangle (30,50);
        if (r1.isequal(r2))    // The contents of objects are compared
            System.out.println("The objects are equal");
        else
            System.out.println("The objects are different");
    }
}
```

The objects are equal

Example:

```
public class Main
{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle (30,50);
        Rectangle r2 = new Rectangle (30,90);
        if (r1.isequal(r2))    // The contents of objects are compared
            System.out.println("The objects are equal");
        else
            System.out.println("The objects are different");
    }
}
```

The objects are different

Example:

```
public class Main
{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle (30,50);
        Rectangle r2 = new Rectangle (30,90);
        r2 = r1;
        if (r2 == r1)
            System.out.println("The objects are equal");
        else
            System.out.println("The objects are different");
    }
}
```

The objects are equal

Example: (Reference only Copy)

```
public class Main
{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle (30,50);
        Rectangle r2 = new Rectangle (30,90);
        System.out.println(r2.getwidth());
        r2 = r1;
        System.out.println(r2.getwidth());
        r2.setlength(70);
        System.out.println(r1.getlength());
    }
}
```

90.0

50.0

70.0

4- Copying an object to another object:

Methods that copy objects:

- There are two ways to copy an object:

1- Reference only copy:

- This is simply copying the address of an object into another reference variable
- EX: `r2 = r1;`

2- Deep Copy (correct):

- This involves creating a new instance of the class and copying the values from one object into the new object.
- This is done using **copy constructor**.

Copy Constructors:

- A copy constructor accepts an existing object of the same class and clones it.

Example: (Copy Constructor)

```
public class Rectangle{  
    private double length;  
    private double width;  
    public Rectangle ()  
    { length = 10;  
      width = 15;  
    }  
    public Rectangle (double l , double w)  
    {  
        length = l;  
        width = w;  
    }  
}
```

```
public Rectangle (Rectangle r)  
{  
    length = r.length;  
    width  = r.width;  
}
```

Example:

```
public class Main
{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle (30,50);
        Rectangle r2 = new Rectangle (r1);
        System.out.println(r2.getLength());
        System.out.println(r2.getWidth());
    }
}
```

30.0

50.0